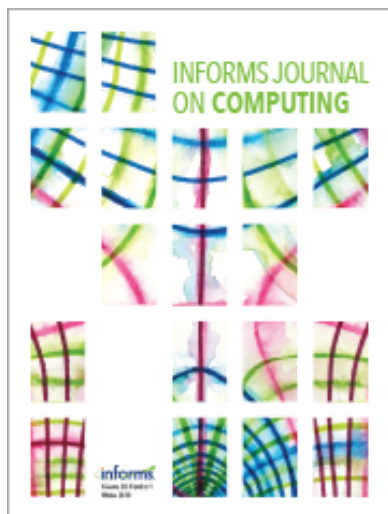




INFORMS Journal on Computing

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

MultiObjectiveAlgorithms.jl: A Julia Package for Solving Multiobjective Optimization Problems

Oscar Dowson, Xavier Gandibleux, Gökhan Kof

To cite this article:

Oscar Dowson, Xavier Gandibleux, Gökhan Kof (2026) MultiObjectiveAlgorithms.jl: A Julia Package for Solving Multiobjective Optimization Problems. INFORMS Journal on Computing

Published online in Articles in Advance 26 Jun 2026

. <https://doi.org/10.1287/ijoc.2025.1449>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2026, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes. For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

MultiObjectiveAlgorithms.jl: A Julia Package for Solving Multiobjective Optimization Problems

Oscar Dowson,^{a,*} Xavier Gandibleux,^b Gökhan Kof^c

^aDowson Farms, Auckland 1061, New Zealand; ^bNantes Université, 44300 Nantes, France; ^cGraduate School of Sciences and Engineering, Koç University, Istanbul 34450, Türkiye

*Corresponding author

Contact: oscar@dowsonfarms.co.nz,  <https://orcid.org/0000-0003-1575-668X> (OD); xavier.gandibleux@univ-nantes.fr (XG); kofgokhan@gmail.com (GK)

Received: June 9, 2025

Revised: December 11, 2025; May 3, 2026

Accepted: May 16, 2026

Published Online in Articles in Advance:
June 26, 2026

<https://doi.org/10.1287/ijoc.2025.1449>

Copyright: © 2026 INFORMS

Abstract. We present MultiObjectiveAlgorithms.jl, an open-source Julia library for solving multiobjective optimization problems written in JuMP. MultiObjectiveAlgorithms.jl implements 10 different solution algorithms that all rely on an iterative scalarization of the problem from a multiobjective optimization problem to a sequence of single-objective subproblems. As part of this work, we extended JuMP to support vector-valued objective functions. Because it is based on JuMP, MultiObjectiveAlgorithms.jl can use a wide variety of commercial and open-source solvers to solve the single-objective subproblems, and it supports problem classes including linear, integer, conic, semidefinite, and general nonlinear.

History: Accepted by Ted Ralphs, Area Editor for Software Tools.

Supplemental Material: The software that supports the findings of this study is available within the paper and its Supplemental Information (<https://pubsonline.informs.org/doi/suppl/10.1287/ijoc.2025.1449>) as well as from the IJOC GitHub software repository (<https://github.com/INFORMSJoC/2025.1449>). The complete IJOC Software and Data Repository is available at <https://informsjoc.github.io/>.

Keywords: optimization • multiobjective • Julia • JuMP

1. Introduction

This paper presents MultiObjectiveAlgorithms.jl (MOA.jl), an open-source Julia (Bezanson et al. 2017) library for solving multiple objective optimization models written in JuMP (Lubin et al. 2023). Multiobjective optimization programs are well-studied in the literature; see, for example, Ehrgott and Gandibleux (2000), Ehrgott (2005), Emmerich and Deutz (2018), Ehrgott et al. (2026), and the references therein. We formulate a multiobjective optimization problem as

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f_0(\mathbf{x}) \\ \text{s.t.} \quad & f_i(\mathbf{x}) \in S_i, \quad i = 1, \dots, m, \end{aligned} \tag{1}$$

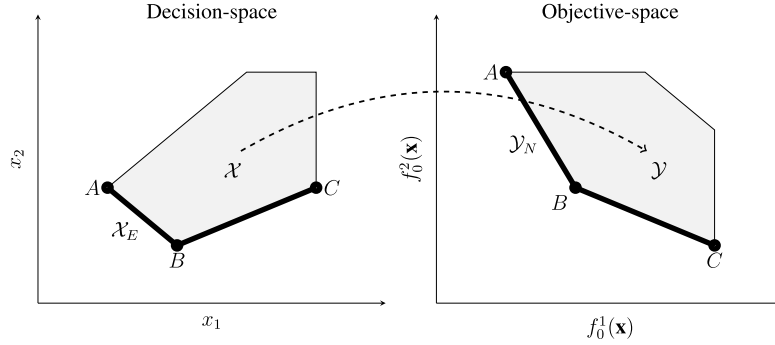
where $f_0 : \mathbb{R}^n \rightarrow \mathbb{R}^{p_0}$, where $p_0 \geq 2$, and $f_i : \mathbb{R}^n \rightarrow \mathbb{R}^{p_i}$, where $p_i \geq 1$, and $S_i \subset \mathbb{R}^{p_i}$ for $i = 1, \dots, m$. The functions f and sets S abstract over the various classes of mathematical optimization problems from mixed-integer linear to semidefinite programs. For more details on this standard form, see Legat et al. (2021).

1.1. Solutions

Let $\mathcal{X}$ denote the set of all feasible solutions of (1) and let $\mathcal{Y} = \{f_0(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ denote the set of corresponding objective vectors. The set $\mathcal{X}$ belongs to decision space, and the set $\mathcal{Y}$ belongs to objective space.

Unlike a single-objective program, in which an optimal solution is reported via a single vector for $\mathbf{x}$, a multiobjective program replaces the notion of optimality with the notion of efficiency (Ehrgott 2005). Let $\leq$ be defined for $\mathbf{a}, \mathbf{b} \in \mathbb{R}^{p_0}$ such that $\mathbf{a} \leq \mathbf{b} \iff \mathbf{b} - \mathbf{a} \in \mathbb{R}_+^{p_0}$ and $\mathbf{a} \neq \mathbf{b}$. Consider $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$. If $f_0(\mathbf{x}_1) \leq f_0(\mathbf{x}_2)$, we say $\mathbf{x}_1$ dominates $\mathbf{x}_2$ and $f_0(\mathbf{x}_1)$ dominates $f_0(\mathbf{x}_2)$. A feasible solution $\mathbf{x} \in \mathcal{X}$ is an efficient solution if there are no feasible solutions that dominate it. The corresponding objective vector, $\mathbf{y} = f_0(\mathbf{x})$, is called a nondominated point. A complete set of efficient solutions is denoted $\mathcal{X}_E \subseteq \mathcal{X}$, and the set of all nondominated points $\mathcal{Y}_N = \{f_0(\mathbf{x}) : \mathbf{x} \in \mathcal{X}_E\}$ is called the nondominated set. A visualization of decision and objective space is provided in Figure 1.

Figure 1. Visualization of Decision and Objective Space for a Linear Program with Two Decision Variables and Two Objectives to Minimize



Notes. The feasible set $\mathcal{X}$ in decision space is projected to the set $\mathcal{Y}$ in objective space via the objective function. The solid line A-B-C in decision space is a complete set of efficient solutions $\mathcal{X}_E$. The solid line A-B-C in objective space is the corresponding nondominated set $\mathcal{Y}_N$.

Because of the existence of equivalent solutions, that is, solutions $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}_E$ such that $f_0(\mathbf{x}_1) = f_0(\mathbf{x}_2)$, the definition of $\mathcal{X}_E$ is refined: the maximum complete set $\mathcal{X}_E$ contains all solutions $\mathbf{x}$ corresponding to all nondominated points $\mathbf{y} = f_0(\mathbf{x})$ and may contain multiple $\mathbf{x}$ for the same $\mathbf{y}$; a complete set $\mathcal{X}_E$ contains at least one $\mathbf{x}$ for each nondominated $\mathbf{y}$; and a minimum complete set $\mathcal{X}_E$ contains exactly one $\mathbf{x}$ for each $\mathbf{y}$. Each of these sets may be infinite, for example, in the case of a multiobjective linear program in which the nondominated set is piecewise linear. For this paper, we are concerned only with the minimum complete set $\mathcal{X}_E$.

Some algorithms for solving Problem (1) may be further restricted to finding only a representative set of efficient solutions $\mathcal{X}_R \subseteq \mathcal{X}_E$. A singleton representative set is a special case when $|\mathcal{X}_R| = 1$. Another special case concerns the notion of a supported nondominated point. A nondominated point $\mathbf{y}$ is supported if $\exists \mathbf{w} > 0$ for which the corresponding efficient solution $\mathbf{x}$ is an optimal solution of $\min_{\mathbf{x} \in \mathcal{X}} \mathbf{w}^\top f_0(\mathbf{x})$; otherwise, it is nonsupported. Some algorithms compute only a minimum supported set of efficient solutions $\mathcal{X}_{SE} \subseteq \mathcal{X}_E$, which is the subset of a minimum complete set such that all corresponding nondominated points are supported. The set of solutions computed by an algorithm also depends on the problem type. For example, some algorithms may compute a minimum supported set of efficient solutions for continuous linear programs but a representative set of efficient solutions for integer or nonlinear programs. Other algorithms are applicable only for particular problem classes; for example, they may require that all decision variables are discrete.

1.2. Solution Algorithms

Large numbers of solution algorithms have been proposed for Problem (1). These algorithms can be divided into four groups along two axes according to the nature of solutions returned: algorithms computing exact or approximated solutions and algorithms with or without exploitation of preference information from the decision maker. MOA.jl implements exact algorithms without preference exploitation based on mathematical programming.

At a high level, the algorithms in MOA.jl solve a sequence of subproblems that are variations of the following scalar-objective problem:

$$\begin{aligned} SP(\mathbf{u}, \mathbf{v}) = \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{u}^\top f_0(\mathbf{x}) \\ \text{s.t.} \quad & f_i(\mathbf{x}) \in S_i, \quad i = 1, \dots, m \\ & f_0(\mathbf{x}) \leq \mathbf{v}, \end{aligned} \quad (2)$$

where $\mathbf{u} \in \mathbb{R}^{p_0}$ is a weighted-sum vector that combines the vector-valued objective into a scalar objective, and the vector $\mathbf{v} \in \mathbb{R}^{p_0}$ places an upper bound constraint on the objective values. By iteratively changing $\mathbf{u}$ and $\mathbf{v}$, algorithms can find a minimum complete set. Some algorithms may add additional variables and constraints on $\mathbf{x}$.

MOA.jl is a meta-solver. In high-level Julia code, it modifies the vectors $\mathbf{u}$ and $\mathbf{v}$, and it uses third-party solvers to solve the resulting single-objective subproblems. Because MOA.jl is built on JuMP, it can use a wide variety of commercial and open-source solvers to solve the single-objective subproblems, and it supports problem classes including linear, integer, conic, semidefinite, and general nonlinear. The solutions that MOA.jl returns, although subject to various theoretical guarantees, the solutions that MOA.jl returns depend on the chosen algorithm and problem structure.

As an example, consider the epsilon-constraint algorithm (Haimes et al. 1971). Pseudo-code for solving the biobjective case ($p_0 = 2$) of Problem (1) is provided in Algorithm 1. The function SP means solve Problem (2) and return the optimal objective value. The function `filter_dominated` means remove all elements of the set S that

are dominated by another element. The algorithm is controlled by a scalar parameter $\varepsilon > 0$. When the objective functions can take only discrete integer values and $\varepsilon = 1$, the algorithm is guaranteed to find a minimum complete set, otherwise, the solution is a representative set.

Algorithm 1 (Pseudo-Code for the Biobjective Epsilon-Constraint Algorithm)

```

 $opt_1^* \leftarrow \text{SP}([1, 0], [\infty, \infty])$ 
 $opt_2^* \leftarrow \text{SP}([0, 1], [\infty, \infty])$ 
 $S \leftarrow \{\}, ub_1 \leftarrow opt_1^*, ub_2 \leftarrow \infty$ 
while  $ub_2 > opt_2^*$  do
     $ub_2 \leftarrow \text{SP}([0, 1], [ub_1, \infty])$  and store optimal  $x^*$ 
     $ub_1 \leftarrow ub_1 + \varepsilon$ 
     $S \leftarrow S \cup \{(x^*, f_0(x^*))\}$ 
end
return filter_dominated( $S$ )

```

1.3. Contribution and Outline

The main contribution of this paper is the Julia library `MultiObjectiveAlgorithms.jl`. A novel feature of `MOA.jl` is its modular ability to add new solution algorithms. This makes it easy for researchers to develop new solution algorithms and compare them against the existing state of the art. To enable `MOA.jl`, we extended JuMP to support vector-valued objective functions. The ease with which users can now model, solve, and analyze multiobjective optimization problems is a significant contribution of our work.

The purpose of this paper is not to provide a mathematical description of the various algorithms that are implemented in `MOA.jl`. Instead, our contribution is to make a variety of existing algorithms available in a high-quality open-source library. We refer readers interested in the algorithmic details to the extant literature (see Table 1) and, more practically, to the code archive at Dowson et al. (2026).

As a secondary contribution, we explain our design goals and provide a comparison with alternative software. Our hope is that our positive experiences developing `MOA.jl` provide motivation and inspiration for developers to add similar features in other algebraic modeling languages.

The rest of this paper is organized as follows. In Section 2, we briefly introduce `MOA.jl` by way of a code example. In Section 3, we provide some brief numerical benchmarks. In Section 4, we provide a comparison of `MOA.jl` to related work. Section 5 presents our design principles before we conclude in Section 6.

2. Code Example

As a didactic example of the usage of `MOA.jl`, we consider a knapsack problem with two linear objectives, one constraint, and binary variables:

$$\begin{aligned}
 &\max_{x \in \mathbb{R}^n} Cx \\
 &\text{s.t. } w^\top x \leq b \\
 &\quad x_i \in \{0, 1\}, \quad i = 1, \dots, n,
 \end{aligned} \tag{3}$$

Table 1. Summary of the Algorithms Supported by `MOA.jl`

Algorithm	Citation	Restriction	Lines	Solution
Chalmet	Chalmet et al. (1986)	$p_0 = 2$	112	Minimum complete set
Dichotomy	Aneja and Nair (1979)	$p_0 = 2$	95	Minimum supported set
DominguezRios	Dominguez-Rios et al. (2021)	IP	220	Minimum complete set
EpsilonConstraint	Haimes et al. (1971)	$p_0 = 2$	101	Minimum complete set
Hierarchical	Gurobi Optimization, LLC (2025b)		92	Singleton representative set
KirlikSayin	Kirlik and Sayin (2014)	IP	141	Minimum complete set
Lexicographic	Isermann (1982)		111	Representative set
RandomWeighting	GAMS Software GmbH (2025)		61	Representative set
Sandwiching	Koenen et al. (2023)		128	Minimum supported set
TambyVanderpooten	Tamby and Vanderpooten (2021)	IP	231	Minimum complete set

Notes. The Restriction column indicates if the algorithm is restricted to a particular problem class: $p_0 = 2$ means the algorithm is restricted to biobjective problems, IP means the algorithm is restricted to pure integer linear subproblems. “Lines” is the number of lines of Julia code required to implement each algorithm. The “Solution” column assumes the problem is pure linear or pure integer; for other problem classes, the set of solutions may be different.

where C is a 2-by- n matrix of costs, w is a vector of length n for the weight of each item, and b is the capacity of the knapsack. The code to implement this problem in JuMP is

```

1  using JuMP, HiGHS
2  import MultiObjectiveAlgorithms as MOA
3  function solve_knapsack(C::Matrix, w::Vector, b::Real, algorithm)
4      @assert size(C) == (2, length(w))
5      model = Model{() }-> MOA.Optimizer{HiGHS.Optimizer}()
6      set_silent(model)
7      set_attribute(model, MOA.Algorithm(), algorithm)
8      @variable(model, x[1:length(w)], Bin)
9      @objective(model, Max, C * x)
10     @constraint(model, w' * x <= b)
11     optimize!(model)
12     assert_is_solved_and_feasible(model)
13     return [value.(x; result = i) => objective_value(model; result = i)
14             for i in 1:result_count(model)]
15 end

```

Lines 1 and 2 import the required Julia packages. Line 3 defines a Julia function with the relevant arguments (the corresponding end is at line 15). Line 4 performs a sanity check of the input data. Line 5 creates a new JuMP model with `MOA.jl` as the optimizer. Because `MOA.Optimizer` is a meta-solver, it requires a mathematical programming solver as input. Here, we use the HiGHS solver (Huangfu and Hall 2018), which is available as `HiGHS.Optimizer` in the `HiGHS.jl` Julia package. Line 6 turns off printing. Line 7 controls which algorithm `MOA.jl` uses to solve the problem; Section 2.1 provides a complete list, but examples are `MOA.EpsilonConstraint()` and `MOA.Dichotomy()`. Lines 8–12 define and solve the JuMP model using typical syntax. Note how the objective function on line 9 is a vector. Lines 13 and 14 use JuMP’s support for returning multiple solutions to iterate over the number of available results and return a vector of (x, y) pairs corresponding to an efficient solution and its nondominated point.

The `solve_knapsack` function can be called as follows:

```

julia> C, w, b = rand(1:20, 2, 10), rand(1:20, 10), 50.0

julia> solve_knapsack(C, w, b, MOA.EpsilonConstraint())
4-element Vector{Pair{Vector{Float64}, Vector{Float64}}}:
 [1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0] => [58.0, 64.0]
 [1.0, 0.0, 0.0, 1.0, 1.0, 0.0, 1.0, 1.0, 1.0, 0.0] => [65.0, 59.0]
 [1.0, 0.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0] => [69.0, 56.0]
 [1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0] => [71.0, 42.0]

julia> solve_knapsack(C, w, b, MOA.Lexicographic())
2-element Vector{Pair{Vector{Float64}, Vector{Float64}}}:
 [1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0] => [58.0, 64.0]
 [1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0] => [71.0, 42.0]

```

Note how the size of the solution set is algorithm-dependent. Moreover, for all algorithms, the solution returned by `MOA.jl` is a finite set of points. Thus, no algorithm can return a complete set of efficient solutions if variables are continuous. The decision to return only a finite set of points is a significant trade-off that we make for practical reasons. For more detail on this decision, see Section 5.2.

2.1. Supported Algorithms

Table 1 summarizes the algorithms supported by MOA.jl. There are currently 10 algorithms implemented. Some algorithms, such as Chalmet, are specialized for a class of biobjective problems. Others, such as DominquezRios support higher dimensional mixed-integer programming (MIP) problems. The Hierarchical algorithm is identical to the algorithms implemented in solvers such as HiGHS (Huangfu and Hall 2018). We based our implementation on the description in Gurobi Optimization, LLC (2025b). The RandomWeighting algorithm is an iterative sampling algorithm for finding a representative set. We include it in MOA.jl because it appears in GAMS (GAMS Software GmbH 2025).

Each algorithm is relatively simple to implement, requiring less than 250 lines of Julia code. One reason so few lines are required is that we rely heavily on JuMP to manage the complexity of interacting with the single-objective subproblems.

Aside from the supported problem dimension, the biggest difference between the algorithms is the type of solutions that they return. Some algorithms, such as TambyVanderpooten, can return points from a minimum complete set for discrete problems. Other algorithms, such as Hierarchical, return only a single point, and Lexicographic returns a representative set of points corresponding to all lexicographic permutations of the objective vector (this can be disabled to return only a singleton representative set for high-dimensional problems).

3. Benchmarks

In this section, we give a brief set of benchmarks to illustrate how MOA.jl can be used with different algorithms and subproblem solvers. We use the multiobjective binary knapsack instances from Tamby and Vanderpooten (2021). The formulation is the same as Problem (3) except that the objective matrix C is p -by- n instead of 2-by- n . We consider three problem sizes: $(p, n) = (2, 200)$, $(p, n) = (3, 100)$, and $(p, n) = (4, 50)$. For each problem size, we use the 10 problem instances from Tamby and Vanderpooten (2021). We used Julia v1.12.5 (Bezanson et al. 2017), JuMP v1.30.0 (Lubin et al. 2023), and MOA.jl v1.11.0. For the subproblem solvers, we use Gurobi v13.0 (Gurobi Optimization, LLC 2025a) and HiGHS v1.12 (Huangfu and Hall 2018), and we use the Dichotomy, Epsilon-Constraint, and TambyVanderpooten algorithms. When $p > 2$, we study only TambyVanderpooten. For each case, we report the mean over the 10 instances of the number of nondominated solutions found, the number of single-objective subproblems solved, the total runtime in seconds, and the percentage of time spent inside the subproblem solver. All computations were carried out on a 2020 Apple M1 Mac Mini with 16 GB of RAM. We summarize the benchmark results in Table 2. The code and data to reproduce our experiments is available at Dowson et al. (2026).

We verified that our solutions match those reported in Tamby and Vanderpooten (2021); they report the average number of nondominated solutions as 405.4 for $(p, n) = (2, 200)$, 4,090.0 for $(p, n) = (3, 100)$, and 2,240.1 for $(p, n) = (4, 50)$, which matches the results for our exact algorithms. Because the Dichotomy algorithm finds only a minimum supported set, it does not find all nondominated solutions. Interestingly, our solve times are two to five times slower than those reported in Tamby and Vanderpooten (2021) even though we solve a similar number of subproblems. Moreover, the “Time in subproblem, %” column shows that the majority of the total solve time is dominated by the time spent solving the subproblems. This means that Julia, JuMP, and MOA.jl are not bottlenecks in the solution process. Thus, we attribute the runtime differences to either the machine and solver used by Tamby and Vanderpooten (2021) or by solver-specific options that they do not report (we used the default settings).

Table 2. Benchmark Results for the Multiobjective Knapsack Problems of Tamby and Vanderpooten (2021)

p	n	Algorithm	Solver	Solutions	Subproblems	Time, s	Time in subproblem, %
2	200	Dichotomy	Gurobi	34.7	68.9	1	88
2	200	Dichotomy	HiGHS	34.9	68.1	6	98
2	200	EpsilonConstraint	Gurobi	405.4	813.8	29	99
2	200	EpsilonConstraint	HiGHS	405.4	813.8	351	100
2	200	TambyVanderpooten	Gurobi	405.4	818.2	31	99
2	200	TambyVanderpooten	HiGHS	405.4	818.6	366	100
3	100	TambyVanderpooten	Gurobi	4,090.0	14,724.4	673	97
4	50	TambyVanderpooten	Gurobi	2,240.1	20,570.2	1,042	68

Notes. Here, “Solutions” is the size of the nondominated set $|Y_N|$; “Subproblems” is the number of single-objective subproblems that were solved during the algorithm; “Time, s” is the total solve time in seconds; and “Time in subproblem, %” is the percentage of the total solve time that was spent in the solve routine of the subproblems. All values are the mean across the 10 problem instances for each problem size.

For a given problem size and algorithm, we observe differences in the number of subproblems solved by Gurobi and HiGHS (and also between the iteration counts reported by Tamby and Vanderpooten 2021). Moreover, HiGHS finds an additional nondominated solution in instances 9 and 10 of the $(p, n) = (2, 200)$ case with the Dichotomy algorithm. This additional solution is a valid nondominated point, but it is not supported. In theory, the algorithm should not find this point. However, HiGHS finds the nonsupported point because the scalarized objective value is within the MIP gap of a nearby supported nondominated point. Thus, the different number of subproblems explored by each solver is attributable to the different sequences of iterations that can occur when there are multiple near-optimal primal solutions for a particular subproblem.

One surprising result in Table 2 is the order-of-magnitude difference in runtime between HiGHS and Gurobi. Working with the HiGHS developers, we analyzed the instances in more detail. We verified that the performance difference was purely an artifact of the solver and that it was not related to Julia, JuMP, MOA.jl, or how subproblems are efficiently cached and modified between solves. Instead, the reason for the discrepancy is simple: the subproblems are trivial mixed-integer programs with one constraint and 200 (or fewer) decision variables. HiGHS does not attempt to detect and exploit this problem structure; instead, it runs generic techniques that work well on large-scale problems but are relatively expensive on these small-scale problems.

4. Comparison with Related Works

Table 3 compares MOA.jl to related mathematical programming software for modeling and solving multiobjective optimization problems. We compare four modeling language approaches, MOA.jl, AMPL (AMPL Optimization Inc. 2025), MOCVXPY (Diamond and Boyd 2016, Salomon et al. 2025), and GAMS (GAMS Software GmbH 2025), and five open-source solver approaches, BenSOLVE (Löhne and Weißing 2017), HiGHS (Huangfu and Hall 2018), inner (Csirmaz 2021), PaMILO (Böckler et al. 2023), and PolySCIP (Borndörfer et al. 2016).

Many commercial solvers (for example, COPT, CPLEX, Gurobi, Hexaly, and Xpress) have support for multiple objectives. We choose HiGHS as a single representative example of this type of solver because they all implement similar variations of the Lexicographic and Hierarchical algorithms. None supports returning a set of

Table 3. General Comparison of Mathematical Programming Software Packages for Multiobjective Optimization

	Modeling-language-based				Open-source solver-based				
	MOA.jl	AMPL	MOCVXPY	GAMS	BenSOLVE	HiGHS	Inner	PaMILO	PolySCIP
Programming language	Julia	Custom	Python	Custom	C	Many	C	C	C
License	BSD-3	Commercial	Apache	Commercial	GPL-2	MIT	GPL-3	Custom	Custom
Modeling language	JuMP	AMPL	CVXPY	GAMS					Zimpl
Solvers	Many	Many	Many	Many	GLPK	HiGHS	GLPK	CPLEX/Gurobi	SCIP
Multiple solutions	Yes		Yes	Yes	Yes		Yes	Yes	Yes
Problem class									
Linear	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Integer	Yes	Yes		Yes		Yes		Yes	Yes
Nonlinear	Yes	Yes	Convex	Yes				Quadratic	
Algorithms									
Benson			Yes		Yes		Yes		Yes
Böckler								Yes	
Chalmet	Yes								
Dichotomy	Yes								
DominguezRios	Yes								
EpsilonConstraint	Yes			Yes					
Hierarchical	Yes	Yes				Yes			
KirlikSayin	Yes								
Lexicographic	Yes	Yes		Yes		Yes			
RandomWeighting	Yes			Yes					
Sandwiching	Yes			Yes					
TambyVanderpooten	Yes								

Notes. The “Algorithms” are based on the naming scheme in Table 1. The exceptions are Benson (Benson 1998) and Böckler (Böckler and Mutzel 2015). The algorithms implemented by BenSOLVE, inner, MOCVXPY, and PolySCIP are different, but they are all variants of Benson’s. PaMILO and PolySCIP have custom open-source licenses that prevent commercial use.

multiple efficient solutions. To the best of our knowledge, no other popular algebraic modeling languages (e.g., Pyomo; Bynum et al. 2021) support multiobjective problems. MATLAB has a variety of support for multiobjective optimization, including the `paretosearch` function for returning a representative set of linear and non-linear optimization problems (MATLAB 2025). We omit it from Table 3 because we could not confirm what algorithms it implements or what solvers it uses.

Considering the large and well-developed body of literature that exists around multiobjective problems, mathematical programming-based software packages for solving such problems are sparse and relatively feature-poor. When support does exist, most packages focus on returning a single representative solution that is some mix of a lexicographic and hierarchical blending.

The lack of general purpose libraries means that practitioners who want to solve a multiobjective problem and return multiple solutions typically code their own implementation. In our opinion, the lack of a general purpose library for solving multiobjective optimization problems has limited the applicability of multiobjective optimization because it requires practitioners to have a reasonable level of software development proficiency. As a secondary effect, practitioners may implement only a single (and simple) solution algorithm instead of experimenting with a range of more sophisticated solution algorithms.

The lack of general purpose libraries also hurts developers of novel solution algorithms. In most cases, papers that develop new general purpose algorithms publish reproducible source code, but the code is often problem-specific and not intended for general purpose use. If future authors implement their algorithm in `MOA.jl`, they immediately gain the ability to model and solve problems using JuMP, and their work is also available to the wider community. We see this point as a positive, if minor, contribution of our work.

5. Design Principles

We based the design of `MOA.jl` on the following design principles.

5.1. Modular Algorithms

The last decade has led to a proliferation of novel solution algorithms for multiobjective optimization problems. We point to the work of Dominguez-Rios et al. (2021) and Tamby and Vanderpooten (2021) as examples. Because algorithm design and comparison is an ongoing research challenge, we wanted a design that makes writing new algorithms easy. As a secondary consideration, we wanted to make switching between algorithms simple to enable speed and solution quality comparisons between algorithms.

We achieve these goals by leveraging Julia's strength for multiple dispatch. Each algorithm subtypes the `MOA.AbstractAlgorithm` type and implements a single function, `MOA.minimize_multiobjective`. The return value is a termination status and a list of solution points (if they exist). For example, a new algorithm could be implemented as

```
import MultiObjectiveAlgorithms as MOA
struct NewAlgorithm <: MOA.AbstractAlgorithm end
function MOA.minimize_multiobjective(model::MOA.Optimizer, ::NewAlgorithm)
    solutions = MOA.SolutionPoint[]
    # Implement the algorithm using `model.f::MOI.AbstractVectorFunction`
    if error
        return MOI.OTHER_ERROR, nothing
    end
    return MOI.OPTIMAL, solutions
end
```

This algorithm can then be immediately used from JuMP with

```
set_attribute(model, MOA.Algorithm(), NewAlgorithm())
```

Behind the scenes, `MOA.jl` builds the problem, converts from maximization to minimization (if necessary), and converts the returned vector of solutions into the form required by JuMP. The separation of algorithm and modeling interface simplifies writing an algorithm; each of the algorithms implemented in `MOA.jl` is less than 250 lines of code.

5.2. A Solution Is a Finite Set of Points

Unlike the single-objective optimization literature, the multiobjective algorithm literature has not converged on a single definition of what constitutes a solution. As described in Section 1.1, there are at least three common definitions: the maximum complete set, a complete set, and a minimum complete set. These sets may be finite and discrete or continuous and infinite.

In `MOA.jl`, we choose to ignore this complexity. We do not require that an algorithm returns a minimum complete set. We define a solution as a finite set of vectors $\mathbf{x}$ with corresponding objective vectors $\mathbf{y} = f_0(\mathbf{x})$. We order solutions lexicographically based on the objective vector and the objective sense; the first solution is best. We make no quality assertions about the returned list other than that each objective vector is nondominated. We make this decision for pragmatic reasons: it makes the library simpler to implement, and it provides most users with most of what they want in practice, which is the ability to find a representative set of diverse solutions to their problem. The trade-off is that it requires users to have some theoretical knowledge to choose an appropriate solution algorithm for their problem type and to understand whether the returned solution is a representative set or a minimum complete set.

5.3. A Single Objective Sense and a Vector-Valued Objective

When designing `MOA.jl`, we had robust discussions about two alternate approaches: is a multiobjective optimization problem defined by a set of independent scalar objectives, each of which may have a different objective sense, or is it a vector-valued function with a single objective sense? Many users would prefer the former because it allows them to, for example, minimize one objective and maximize another. The latter requires them to normalize all scalar objectives into the same objective sense by multiplying maximization objectives by -1 .

Because `MOA.jl` is based on JuMP, which itself relies on the MathOptInterface standard form (Legat et al. 2021), we chose to implement the library as a single objective sense and a vector-valued objective function. Although more restrictive from a user perspective, this significantly simplified the implementation of `MOA.jl`, and we have not found this decision to be a burden in practice. Note that our decision is purely an artifact of our decision to build on JuMP; GAMS and MOCVXPY, for example, allow a different objective sense for each scalar objective.

6. Conclusions

This paper has presented `MOA.jl`, an open-source Julia package for solving multiobjective optimization problems written in JuMP. Readers are directed to Dowson et al. (2026) for the archived source code and to <https://github.com/jump-dev/MultiObjectiveAlgorithms.jl> for the latest version.

`MOA.jl` implements 10 different solution algorithms, which all rely on an iterative scalarization of the problem from a multiobjective optimization problem to a sequence of single-objective subproblems. Each algorithm is relatively simple to implement, requiring less than 250 lines of code, and the modular design of the library makes it easy to add new algorithms. Because it is based on JuMP, `MOA.jl` can use a wide variety of commercial and open-source solvers to solve the single-objective subproblems, and it supports problem classes ranging from linear to conic, semidefinite, and general nonlinear.

The only gap between recent advances in algorithms for multiobjective optimization and the current design of the library is our decision to return a finite set of points as the solution. As future work, we want to extend `MOA.jl` to support solutions such as nondominated facets and the associated open and closed points. The correct implementation of this is an open question, particularly with how to represent these solutions in JuMP.

To conclude, we hope that our positive experience developing `MOA.jl` serves as motivation and design inspiration for others to add similar features to algebraic modeling languages that do not currently support multiobjective optimization problems.

Acknowledgments

`MOA.jl` is the successor to the `vOptSolver` project (Gandibleux et al. 2017). The development of `vOptSolver` started in the ANR/DFG-14-CE35-0034-01 research project `vOpt` (2015–2019).

References

- AMPL Optimization Inc. (2025) Multiple objective. Accessed June 21, 2025, <https://mp.ampl.com/modeling-mo.html>.
- Aneja Y, Nair K (1979) Bicriteria transportation problem. *Management Sci.* 25(1):73–78.
- Benson H (1998) An outer approximation algorithm for generating all efficient extreme points in the outcome set of a multiple objective linear programming problem. *J. Global Optim.* 13:1–24.
- Bezanson J, Edelman A, Karpinski S, Shah VB (2017) Julia: A fresh approach to numerical computing. *SIAM Rev.* 59(1):65–98.

- Böckler F, Mutzel P (2015) Output-sensitive algorithms for enumerating the extreme nondominated points of multiobjective combinatorial optimization problems. Bansal N, Finocchi I, eds. *Algorithms—ESA 2015* (Springer, Berlin, Heidelberg), 288–299.
- Böckler F, Nemesch L, Wagner MH (2023) PaMILO: A solver for multi-objective mixed integer linear optimization and beyond. Grothe O, Nickel S, Rebennack S, Stein O, eds. *Oper. Res. Proc. 2022, Lecture Notes in Operations Research* (Springer, Cham).
- Borndörfer R, Schenker S, Kutella M, Strunk T (2016) PolySCIP. Greuel G-M, Koch T, Paule P, Sommese AJ, eds. *Math. Software—ICMS 2016, Lecture Notes in Computer Science*, vol. 9725 (Springer, Cham, Switzerland), 228–234.
- Bynum ML, Hachebeil GA, Hart WE, Laird CD, Nicholson BL, Sirola JD, Watson JP, Woodruff DL (2021) *Pyomo—Optimization Modeling in Python*, vol. 67, 3rd ed. (Springer Science & Business Media, Cham, Switzerland).
- Chalmet L, Lemonidis L, Elzinga D (1986) An algorithm for the bi-criterion integer programming problem. *Eur. J. Oper. Res.* 25(2):292–300.
- Csirmaz L (2021) Inner approximation algorithm for solving linear multiobjective optimization problems. *Optimization* 70(7):1487–1511.
- Diamond S, Boyd S (2016) CVXPY: A Python-embedded modeling language for convex optimization. *J. Machine Learn. Res.* 17(1):2909–2913.
- Dominguez-Rios M, Chicano F, Alba E (2021) Effective anytime algorithm for multiobjective combinatorial optimization problems. *Inform. Sci.* 565(7):210–228.
- Dowson O, Gandibleux X, Kof G (2026) MultiObjectiveAlgorithms.jl: A Julia package for solving multi-objective optimization problems. <https://doi.org/10.1287/ijoc.2025.1449.cd>, <https://github.com/INFORMSJoC/2025.1449>.
- Ehrgott M (2005) *Multicriteria Optimization*, 2nd ed. (Springer, Berlin).
- Ehrgott M, Gandibleux X (2000) A survey and annotated bibliography of multiobjective combinatorial optimization. *OR Spektrum* 22(4):425–460.
- Ehrgott M, Köksalan M, Kadziński M, Deb K (2026) Fifty years of multi-objective optimization and decision-making: From mathematical programming to evolutionary computation. *Eur. J. Oper. Res.* 330(1):1–25.
- Emmerich M, Deutz A (2018) A tutorial on multiobjective optimization: Fundamentals and evolutionary methods. *Natural Comput.* 17(3):585–609.
- GAMS Software GmbH (2025) Multi-objective optimization. Accessed May 21, 2025, https://www.gams.com/50/docs/T_LIBINCLUDE_MOO.html.
- Gandibleux X, Soleilhac G, Przybylski A, Ruzika S (2017) vOptSolver: An open source software environment for multiobjective mathematical optimization. *IFORS2017: 21st Conf. Internat. Federation Oper. Res. Societies* (International Federation of Operational Research Societies, Quebec).
- Gurobi Optimization, LLC (2025a) Gurobi optimizer. Accessed May 21, 2025, <https://www.gurobi.com>.
- Gurobi Optimization, LLC (2025b) Multiple objectives. Accessed May 21, 2025, <https://docs.gurobi.com/projects/optimizer/en/current/features/multiobjective.html>.
- Haimes Y, Lisdon L, Wismer D (1971) On a bicriterion formulation of the problems of integrated system identification and system optimization. *IEEE Trans. Systems Man Cybernetics* 3:296–297.
- Huangfu Q, Hall JAJ (2018) Parallelizing the dual revised simplex method. *Math. Programming Comput.* 10(1):119–142.
- Isermann H (1982) Linear lexicographic optimization. *OR Spectrum* 4(4):223–228.
- Kirlik G, Sayin S (2014) A new algorithm for generating all nondominated solutions of multiobjective discrete optimization problems. *Eur. J. Oper. Res.* 232(3):479–488.
- Koenen M, Balvert M, Fleuren H (2023) A renewed take on weighted sum in sandwich algorithms: Modification of the criterion space. Technical report, Center for Economic Research, Tilburg, Netherlands.
- Legat B, Dowson O, Dias Garcia J, Lubin M (2021) MathOptInterface: A data structure for mathematical optimization problems. *INFORMS J. Comput.* 34(2):672–689.
- Lubin M, Dowson O, Garcia JD, Huchette J, Legat B, Vielma JP (2023) JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Math. Programming Comput.* 15(3):581–589.
- Löhne A, Weißing B (2017) The vector linear program solver bensolve—Notes on theoretical background. *Eur. J. Oper. Res.* 260(3):807–813.
- MATLAB (2025) paretosearch—Find points in pareto set. Accessed June 29, 2025, <https://au.mathworks.com/help/gads/paretosearch.html>.
- Salomon L, Dörfler D, Löhne A (2025) MOCVXPY: A CVXPY extension for multiobjective optimization. Preprint, submitted October 23, <https://arxiv.org/abs/2510.21010>.
- Tamby S, Vanderpooten D (2021) Enumeration of the nondominated set of multiobjective discrete optimization problems. *INFORMS J. Comput.* 33(1):72–85.